The British University in Dubai

CSAI-413 Natural Language Processing Applications

Group Project: Design, Investigate, and Evaluate a Retrieval-Augmented AI System (RAG)

This project constitutes 20% of the overall course grade and will be evaluated out of 100 marks.

The evaluation will be divided into three components:

- **Deliverable 1: System Design + Baseline (Report): 30%**
- **Deliverable 2: Experimental Investigation + Evaluation (Report): 50%**
- **Presentation (Viva + Demo): 20%**

Project Overview

This project focuses on the design and critical evaluation of modern AI-powered NLP systems using Retrieval-Augmented Generation (RAG).

You will build an intelligent system that:

- Retrieves relevant information from external knowledge sources
- Uses a language model (LLM) to generate grounded responses
- Critically evaluates system behavior, limitations, and failures

This project does NOT aim to assess your ability to:

- copy code
- use AI tools
- assemble existing pipelines

Instead, it evaluates your ability to understand, analyze, justify, and critically evaluate AI systems.

Use of online resources, libraries, and AI tools is allowed, but all reasoning, analysis, and conclusions must be your own.

Learning Objectives

This project supports the achievement of the CSAI413 module learning outcomes:

- **LO1:** Apply modern NLP and LLM-based techniques
- **LO2:** Evaluate and compare AI systems
- **LO3:** Analyze and define real-world NLP problems

- **LO4:** Collaborate effectively in teams
- **LO5:** Design end-to-end AI systems
- **LO6:** Critically assess AI system behavior and limitations

Project Summary

Topic Selection: Each group will design a domain-specific AI assistant using RAG.

Example domains:

- University knowledge assistant
- Legal or policy documents
- Technical documentation
- Financial or business knowledge
- Healthcare information

Dataset Requirements

- **A document-based knowledge source**
- Sources may include:
  - PDFs, reports, websites, articles
- Must:
  - be clearly described
  - require meaningful retrieval (not trivial QA)

Core System Components

All projects must include:

- Document preprocessing and chunking
- Text embeddings and indexing
- Retrieval mechanism
- LLM-based response generation
- Evaluation pipeline

R1. Deliverable 1: System Design + Baseline (30%)

Deadline: Week 6 – 22th May 2026 : Report (PDF) + Code + Evidence

Required Components

1. Problem Definition

- Domain and use case
- Target users
- Why RAG is appropriate (vs direct LLM)

2. Dataset Description

- Source and structure
- Preprocessing steps
- Chunking strategy (with justification)

3. System Architecture

- Diagram of the full pipeline
- Description of each component

4. Design Justification

- Why this embedding method?
- Why this retrieval approach?
- What design alternatives were considered?

5. Baseline Implementation

- Working RAG system
- Basic retrieval + generation

6. Initial Evaluation

- Example queries
- Observed outputs
- Early failure cases

Evidence Required

- Screenshots of:
    - system outputs
    - retrieval results
    - intermediate steps
- Code snippets (explained, not just pasted)

R2. Deliverable 2: Experimental Investigation + Evaluation (50%)

Deadline: Week 9 – 12th June 2026 : Report (PDF) + Code + Evidence

Required Components

1. System Improvements

- Modifications to:
    - retrieval
    - chunking
    - prompts
    - architecture

2. Hypothesis-Driven Experiments

Define and test hypotheses:

Example: "Smaller chunks improve retrieval accuracy because they reduce noise."

Each experiment must include:

- hypothesis
- setup
- results
- interpretation

3. Controlled Experiments

Systematically vary:

- chunk size
- embedding model
- prompt design
- retrieval strategy

4. Evaluation Framework

You must evaluate:

- relevance of retrieved documents
- answer correctness
- hallucination presence
- robustness to difficult queries

5. Failure Analysis

You must identify and classify failures:

- retrieval failure
- hallucination
- ambiguous queries
- irrelevant context

For each:

- example
- explanation
- attempted fix

6. Comparative Analysis

Compare:

- baseline vs improved system OR
- RAG vs direct LLM

7. Results and Insights

- Tables / qualitative analysis
- Explanation of findings

Evidence Required

- Screenshots of:
  - different experiment outputs
  - failure cases
  - system behavior under variations
- Code snippets with explanation

R3. Presentation (Viva + Demo) (20%) – Week 10

Requirements

- Live demonstration of system
- Explanation of:
  - design decisions
  - experiments
  - failures

Individual Understanding Requirement

All students are required to have a **full understanding of the entire project**, including:

- system design and architecture
- implementation details
- experiments conducted
- results and evaluation
- identified failures and improvements

Viva Assessment Policy

The viva (oral examination) will assess each student's individual understanding of the project.

- Each student must answer questions independently
- Questions may cover any part of the project, regardless of individual contribution
- Lack of understanding may result in reduction of individual marks

Students who are unable to explain their project during the viva may receive:

- reduced marks in the presentation component
- additional penalties in overall project grading

Team Size

- 3 to 4 students

Implementation Requirements

- Language: Python (preferred)
- Any libraries/tools allowed

Code Submission

Each team must submit:

- Full source code
- Clear structure and documentation
- Instructions to run the system

Use of Generative AI (GAI) - Mandatory Section in Report:

“AI Usage Disclosure”

- What tools were used
- How they were used
- What was done independently
- Add students name, ID and signature.

Plagiarism Policy

All work must be original. However, the use of external code is allowed **only if you understand, adapt, and justify it**. Failure to demonstrate understanding (especially during viva) will result in penalties.

Bonus Features (Optional)

- Interactive application (e.g., web interface)
- Advanced retrieval (hybrid search)
- Safety mechanisms (hallucination detection)